



Chapter 9: Implementing Notifications

In our professional and personal lives, we perform multiple tasks every day, often relying on a variety of applications to help get them done: messaging apps, mail clients, productivity tools are just a few examples. Continuously checking these tools to see if there's an important update would be very tedious. Therefore, especially with the advent of smartphones, notifications have become critical to manage our digital life. Instead of forcing us to continuously check what's going on, it's the application itself that will notify us when something important is happening and we need to pay attention.

Notifications play a critical role in all kinds of applications, including desktop ones. As such, Windows 10 has introduced (and evolved release after release) a rich notification ecosystem, that we can use to interact with our users even when they aren't actively working with our application. As we're going to learn in this chapter, Windows doesn't enable only a passive notification ecosystem: you can enrich them with buttons, text boxes, dropdowns, and many other ways to make them interactive.

Windows 10 has also introduced a dedicated area for notification, called **Action Center**, which was redesigned in Windows 11. It's a panel that opens up on the right of the screen when you click on the time and date, which acts as an inbox for all the notifications generated by the installed applications. Even if you miss a notification because you weren't in front of your computer when it was displayed, you'll be able to find it in this section.

In this chapter, we'll address the following topics:

- Sending notification from a Windows application
- Working with toast notifications
- Adding interaction to a toast notification
- Displaying a progress bar
- Displaying a badge in the taskbar
- Implementing push notifications

By the end of this chapter, you will have an understanding of the available notification types, how to integrate them, and how to send them. Let's start!

Technical requirements

The code for the chapter can be found here:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter09>

Sending a notification from a Windows application

Notifications are another one of the features which are part of the **Universal Windows Platform (UWP)** ecosystem and that, originally, wasn't available to the Win32 ecosystem. However, expanding this feature to the whole developer ecosystem very soon became a priority for Microsoft, since more and more apps wanted to participate in the notification ecosystem to take advantage of all the new features and to provide a more streamlined experience. When you participate in the native ecosystem, rather than building your own custom notification solution, you get advantages like the following:

- A familiar look and feel for the user, which follows the Windows UX guidelines
- Integration with features such as **Focus Assist**, which automatically hides all the notifications when your screen is mirrored or when you're sharing your screen with other users through applications such as Microsoft Teams
- Integration with the Action Center, so that users can review notifications even if they were sent while they were away from the computer

Therefore, even before Microsoft started to work on the Windows App SDK, the team decided to provide helpers in the Windows Community Toolkit, which makes it easier to add notifications to any kind of application, either built with UWP, WPF, Windows Forms, or WinUI. The Windows Community Toolkit also made it easier to work with the notification payloads. The content of a notification, in fact, is described with an XML payload to better support push notification scenarios, in which notifications aren't generated locally by the Windows application but can be generated, for example, from a backend written in Java, Node.js, or Python. However, when it comes to local notifications, manipulating XML introduces some challenges, since it isn't a strongly typed language. The Windows Community Toolkit, instead, offers APIs that will generate the XML for you, enabling you to use C# objects to define the notification's content.

Windows supports multiple types of notifications:

- **Toast notifications:** These are the most common ones. They are rendered as a popup displayed at the bottom right corner of the screen. They are the richest ones since they can include text, images, actions, and so on. Toast notifications are moved to the Action Center if the user does not dismiss them.
- **Badge notifications:** These are the ones you can use to display and update a counter on the application's icon. They are used, for ex-

ample, by messaging applications to display the number of unread messages directly in the Taskbar.

- **Raw notifications:** These contain any kind of payload, and, as such, it's up to your application to manually parse the content and handle them. Raw notifications are usually coupled with push notifications, which are generated by an online service (such as a web application or a job running in the cloud) and then handled by your local application. We will cover this topic in this chapter only very briefly.
- **Tile notifications:** Windows 10 offers the concept of tiles, which are special icons for your applications that you could pin to the Start menu. With tiles notifications, an application can update the icon to display relevant information without asking the user to open it. An example is the built-in Weather application, which can update the tile with the forecast for the current location. Tiles, however, are now considered deprecated since they have been removed in Windows 11. As such, we won't cover them in this chapter.

In this chapter, we're going to use a WinUI application as an example, but the same exact set of APIs works in a **WPF** or **Windows Forms application**). The only difference in the way you use the features comes from the deployment model: if you choose to distribute your application as packaged with MSIX, in fact, there will be an extra step to make.

Let's start!

Working with toast notifications

Regardless of the platform and the deployment model of your choice, the first step is to install the Windows Community Toolkit package dedicated to notifications. Right-click on your project, choose **Manage**

NuGet packages, and look for the package with **CommunityToolkit.WinUI.Notifications** as an identifier.

NOTE

*Despite the package name containing the **WinUI** keyword, it isn't specific only for this platform, and can work also with WPF, Windows Forms, or UWP.*

The other requirement is to use a dedicated **TargetFramework** for Windows 10 or 11 since the toolkit needs to consume some APIs from the Universal Windows Platform. In our scenario, we're building a WinUI application, so we're good to go. If you're using Windows Forms or WPF, instead, make sure to set the **TargetFramework** at least as follows:

```
<TargetFramework>net6.0-windows10.0.19041.0
</TargetFramework>
```

In the case of a WinUI project, there's also another change to make. By default, when you create a new WinUI application, the minimum supported Windows version is Windows **10 10.0.17763.0**, which is the October 2018 update. This is controlled by the **TargetPlatformVersion** property of the **.csproj** file:

```
<TargetPlatformMinVersion>10.0.17763.0</TargetPlatform
MinVersion>
```

The notification APIs from the Windows Community Toolkit, however, use features that have been introduced in Windows **10 10.0.18362.0**, which is the May 2019 update. If you want to remove the warnings that Visual Studio will give you otherwise, you should change the **TargetPlatformMinVersion** property as follows:

```
<TargetPlatformMinVersion>10.0.18362.0</TargetPlatform
MinVersion>
```

Now you're all set. Let's start with a very basic scenario, which requires the usage of APIs included in the **CommunityToolkit.WinUI.Notifications** namespace:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .Show();
}
```

Simple, isn't it? You just need to create a new **ToastContentBuilder()** object, which exposes different methods to customize the notification's template. In this case, we're using the **AddText()** one to add two text elements. Once we have created the notification payload, we can display it by calling the **Show()** method. Under the hood, the library will generate the following XML payload:

```
<binding template="ToastGeneric">
    <text hint-maxLines="1">New message</text>
    <text>There's a message for you!</text>
</binding>
```

As you can see, the C# APIs are much more convenient to use than having to manually generate the XML content.

If you're going to test this code in an unpackaged scenario (a WPF, Windows Forms, or an unpackaged WinUI app), everything will work. If, instead, you are using a WinUI packaged app or you have added a Windows Application Packaging Project to your solution so you could package your WPF or Windows Forms app, you will get an exception. In a packaged scenario, in fact, we have to register the notification activation in the manifest first. Let's see how to do it.

Supporting toast notifications in packaged apps

To enable notifications in a packaged app, we need to register the **ToastNotificationActivation** extension in the manifest, otherwise, we would get an exception every time we try to display a notification. Additionally, this extension is required to support the activation scenarios we're going to see later in this chapter: the user interacts with the notification, and we want to trigger an action.

Right-click on the **Package.appxmanifest** file of your application (it's either in the main project if it's a WinUI app, or in the Windows Application Packaging Project if it's a WPF or Windows Forms app) and choose **View code**. The extensions we need to add, in fact, aren't supported by the visual editor.

The first step is to declare two custom namespaces in the XML, as in the following sample:

```
<Package
  xmlns:com="http://schemas.microsoft.com/appx/manifest/com/w
    indows10"
  xmlns:desktop="http://schemas.microsoft.com/appx/manife
    st/
      desktop/windows10"
  IgnorableNamespaces="com desktop">
  ...
</Package>
```

Then we must declare two extensions inside the **Application** instance node, as in the following snippet:

```
<Application Id="App"
  Executable="$targetnametoken$.exe"
  EntryPoint="$targetentrypoint$"
  ...
  <Extensions>
    <desktop:Extension Category="windows
```

```

        .toastNotificationActivation">
    <desktop:ToastNotificationActivation
ToastActivator
        CLSID="535A532A-C79C-4B9D-BDE4-FCD0B3E3C171" />
    </desktop:Extension>
    <com:Extension Category="windows.comServer">
        <com:ComServer>
            <com:ExeServer
Executable="WinUINotifications.exe"
            Arguments="-ToastActivated"
DisplayName="Toast
            activator">
                <com:Class Id="535A532A-C79C-4B9D-BDE4-
                    FCD0B3E3C171" DisplayName="Toast
activator"/>
            </com:ExeServer>
        </com:ComServer>
    </com:Extension>
</Extensions>
</Application>

```

From the preceding code, we see that the first extension is called **windows.toastNotificationActivation**. It's a unique identifier for our application, so it's important to replace the **ToastActivatorCLSID** property with a unique GUID. You can generate one from Visual Studio (you'll find the option under the menu **Tools** | **Create GUID**) or using one of the many available online tools, such as

<https://www.guidgenerator.com/>.

The second extension is **windows.comServer** and it's required to register the toast activator as a COM extension so that Windows can invoke it when needed. Compared to the preceding sample, there are two properties to change:

- The **Executable** property of the **ExeServer** entry must match the main executable of your application (which, in most of the cases,

matches with the name of the project).

- The **Id** property of the **Class** entry must match the same GUID that you have previously generated and assigned to the **ToastActivatorCLSID** property.

Thanks to these changes, the exception will go away, and toast notifications will start to work. Let's explore now other ways to enrich a notification.

Adding images

Toast notifications support multiple types of images, which are translated into a different placement inside the template. The simplest scenario is having an image placed on the left of the text content. This approach is also called **app logo** since, on older versions of the operating system, if you didn't specify any image, Windows used the default logo of the application. On the most recent versions, including Windows 11, instead, if you don't specify this option, the image will simply be left out, leaving all the space for the text content. The logo will be displayed, instead, at the top, right before the name of the application that generated the notification.

You can use the **AddAppLogoOverride()** method, passing as a parameter the **Uri** of the image. If your application is packaged, the **Uri** can be an HTTP URI (so an image available on the internet) or an application URI (so an image included in the package). If your application is unpackaged, instead, you can't use an HTTP URI, but you will have to download the image first on your local machine and then create a URI with the full path of the file. The following sample shows all the supported scenarios:

```
private void ShowNotification()
{
    //packaged app, from the network
    new ToastContentBuilder()
```

```
.AddText("New message")
.AddText("There's a new message for you!")
.AddAppLogoOverride(new Uri
    ("https://www.mywebsite.com/myimage.png"))
.Show();
//packaged app, from the package
new ToastContentBuilder()
    .AddText("New message")
    .AddText("There's a new message for you!")
    .AddAppLogoOverride(new Uri("ms-appx:
        ///Assets/myimage.png"))
    .Show();
//unpackaged app, from the disk
//packaged app, from the network
new ToastContentBuilder()
    .AddText("New message")
    .AddText("There's a new message for you!")
    .AddAppLogoOverride(new Uri(@"C:\myimage.png"))
    .Show();
}
```

This is how this type of image is rendered in a notification:

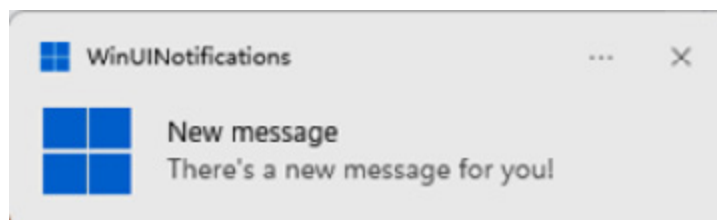


Figure 9.1 – A toast notification with an app logo image

Another choice to display images in a notification is using the concept of a **hero image**. In this scenario, the image is displayed with a bigger size at the top of everything else (including the logo and the name of the application). This feature is enabled by the **AddHeroImage()** method and it has the same requirements we have seen for the logo: if the application is packaged, the image can be either on the internet or

local; if it's unpackaged, instead, it must be local. Let's take a look at the following example:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .AddHeroImage(new Uri
            ("https://www.mywebsite.com/myimage.png"))
        .Show();
}
```

The following screenshot shows the usage of a hero image in a notification:

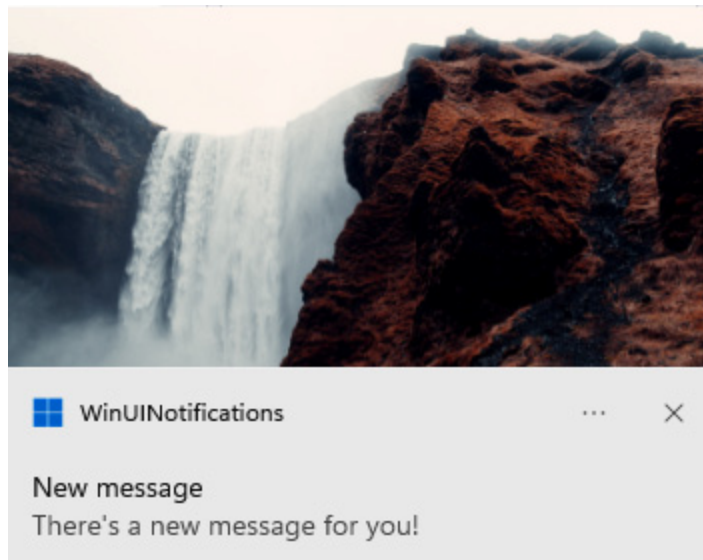


Figure 9.2 – A toast notification with a hero image

Finally, you can also include the image inline, which in this case will be displayed below the main content. This feature works like the others related to images (with the same restrictions), except that the method to use is called **AddInlineImage()**, as in the following sample:

```
private void SendNotification()
{
    new ToastContentBuilder()
```

```
.AddText("New message")  
.AddText("There's a new message for you!")  
.AddInlineImage(new Uri  
    ("https://www.mywebsite.com/myimage.png"))  
.Show();  
}
```

The preceding code will generate the following result:

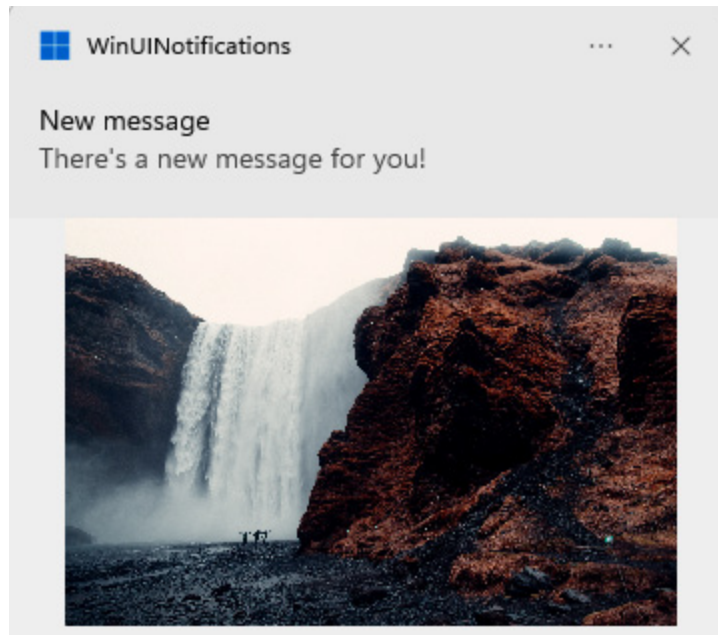


Figure 9.3 – A toast notification with an inline image

Let's see now how to customize the basic information in a notification.

Customizing the application's name and the timestamp

As we have seen in the previous examples, by default, a notification will use, as a header, the logo, and the name of the application that generated it. Another piece of information that is generated automatically is the timestamp: when the notification gets stored in the Action Center, it will display the date and time it was generated.

We can override these two values if needed. For example, our application might have a complex and long name and we want to use a more

friendly one; or the notification is about an event that happened in the past or that it's going to happen in the future.

The name can be changed using the **AddAttributionText()** method, while the timestamp can be changed using the **AddCustomTimeStamp()** one. The following sample shows both in action:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .AddAttributionText("via my amazing WinUI app")
        .AddCustomTimeStamp(new DateTime(2022, 12, 25,
            15, 0, 0))
        .Show();
}
```

All the examples we have seen so far took care of creating a new notification on top of all the existing ones. What if, instead, you want to manipulate the existing notifications created by your application? Let's introduce the concept of tagging.

Tagging a toast notification

Every time your application sends a toast notification, by default it will be automatically added on top of the others. If your application sends multiple notifications, this is what you're going to see in the Action Center:

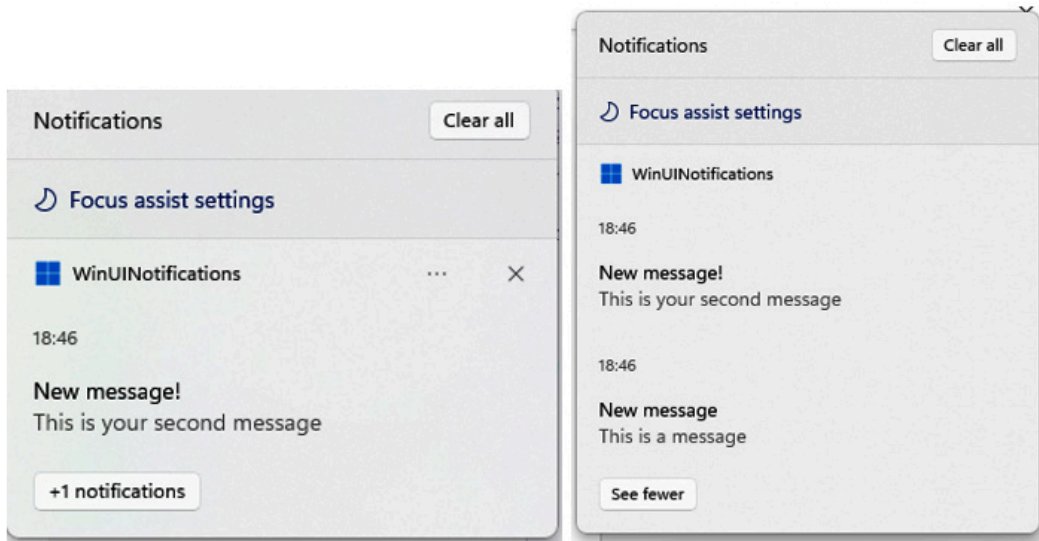


Figure 9.4 – Multiple toast notifications generated by the same applications

Windows will display only the most recent one by default (on the left), but the button below will tell you that there are more notifications available. If you click it, the Action Center will display all of them.

This approach is a great fit, for example, for a messaging app. But what if you're building a football application, which delivers a notification every time the score of the match changes? In this case, retaining all the old notifications might confuse the user. To better manage this scenario, Windows supports the concept of tagging a notification with a keyword. If your application sends a new notification with the same tag, Windows will replace the existing one, rather than adding it on top of the others. This goal is achieved by setting the **Tag** property on the notification:

```
private void SendNotification2()
{
    new ToastContentBuilder()
        .AddText("Goal!")
        .AddText("The new score is 2-0")
        .Show(toast =>
        {
            toast.Tag = "match1";
```

```
});  
}
```

When we call the **Show()** method, we also get a reference to the generated **ToastNotification** object, so that we can further customize other options, including the **Tag** one.

Tagging is important also to better support another scenario: scheduling notifications.

Scheduling a notification

In all the examples we have seen so far we have used the **Show()** method, which immediately triggers the notification. Another option is to schedule them. If you're building a calendar application, for example, you might want to schedule a reminder when the user creates an appointment.

The way you build a scheduled notification is the same as we have seen for the traditional ones. The only difference is that, instead of calling the **Show()** method, you're going to call the **Schedule()** one passing, as a parameter, a **DateTimeOffset** object that represents the date and time when you want to display the notification. The following sample shows scheduling a notification 5 minutes from now:

```
private void SendNotification()  
{  
    new ToastContentBuilder()  
        .AddText("New message")  
        .AddText("This is a message")  
        .Schedule(DateTimeOffset.Now.AddMinutes(5));  
}
```

When you're using the scheduling approach, tagging the notification becomes even more important, since it enables you to retrieve it at a later stage so that we can delete it or change it. In our calendar application, if the user cancels the appointment from the calendar, we don't

want the notification about the event to be still displayed. To tag a scheduled notification, we must use the same approach we have used for a regular one, as in the following sample:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("Christmas party")
        .AddText("Party with family and friends")
        .Schedule(new DateTimeOffset(new DateTime(2021, 12,
25,
        10, 0, 0)), toast =>
        {
            toast.Tag = "event1";
        });
}
```

Thanks to the **ToastNotifierCompat** class, we can then retrieve the list of all the notifications scheduled by our application and, by using the tag, find the one we need to delete:

```
private void DeleteNotification()
{
    ToastNotifierCompat notifier = ToastNotification
        ManagerCompat.CreateToastNotifier();
    IReadOnlyList<ScheduledToastNotification>
        scheduledToasts = notifier.GetScheduledToast
        Notifications();
    var notification = scheduledToasts.FirstOrDefault(i
=>
        i.Tag == "match1");
    if (notification != null)
    {
        notifier.RemoveFromSchedule(notification);
    }
}
```

Once we have created a notifier using the **`ToastNotificationManagerCompat.CreateToastNotifier()`** method, we can use the **`GetScheduledToastNotifications()`** method to retrieve a list of all the scheduled notifications that belong to our application. Then, by using the **`FirstOrDefault()`** expression, we can retrieve the one with our tag (**`match1`**) and pass it as a parameter to the **`RemoveFromSchedule()`** method to delete it.

Now that we have learned the basic ways to customize and manage notifications, let's see one of the most interesting features: supporting interactions.

Adding interaction to a notification

What makes notifications so powerful is that they don't represent just a passive piece of information. The user can interact with them to perform actions without having to open the application. Let's explore the various ways a user can interact with a notification.

The most basic interaction is clicking on the notification, which is also the most common scenario. For example, users get a notification that they have received a new mail and when they click on it, they expect the mail client to open up so that they can read it.

This scenario can be accomplished thanks to the **`AddArgument()`** method, which you can use to pass a key /value pair that you can retrieve when the application is activated from a notification. Let's see the following example:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
```

```
.AddText("There's a new message for you!")  
.AddArgument("messageId", "1983")  
.Show();  
}
```

We have added an argument called **messageId**, with as value the hypothetical ID of the message that was displayed in the notification.

Another scenario is using a notification to enable users to perform multiple actions without having to open the application. If we reuse the example of a mail client, we could allow the user to directly mark a mail as read or flag it as important directly from the notification. In this case, we need to add another layer of interactivity: buttons.

Buttons are represented by the **ToastButton** class and, to each of them, you can assign a different argument, which you can use when the application is activated to understand which button was pressed. Let's see the following example:

```
private void OnSendNotificationWithReply ()  
{  
    new ToastContentBuilder()  
        .AddText("New message")  
        .AddText("There's a new message for you!")  
        .AddButton(new ToastButton()  
            .SetContent("Mark as read")  
            .AddArgument("action", "read")  
            .AddArgument("messageId", "1983"))  
        .AddButton(new ToastButton()  
            .SetContent("Flag as important")  
            .AddArgument("action", "flag")  
            .AddArgument("messageId", "1983"))  
        .Show();  
}
```

This notification includes two buttons, which are added using the **AddButton()** method. We set the label of the button (using the

SetContent() method) and, with the same **AddArgument()** method we have learned in the previous example, we add two arguments that will help us to understand which button was pressed (through the **action** parameter) and which is the message where we must apply the action (through the **messageId** parameter).

The following screenshot shows the result of the preceding code:

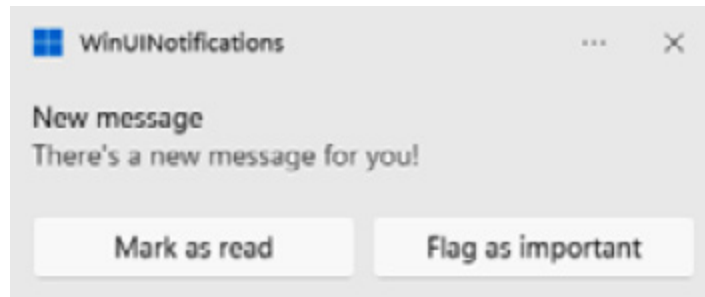


Figure 9.5 – A toast notification with two buttons

Now that our notification can pass activation arguments, how can we retrieve them? Let's learn about that in the next section.

Handling activation from a notification

In [*Chapter 4*](#), *The Windows App SDK for a UWP Developer*, we have learned how we can manage application's activation using the **AppInstance** class. For toast notifications, instead, we must use a different approach, since as we have learned so far this feature isn't part of the Windows App SDK yet, but it's included in the Windows Community Toolkit.

The action is handled through an event called **OnActivated**, which is exposed by the **ToastNotificationManagerCompat** class included **CommunityToolkit.WinUI.Notifications** namespace. This event must be subscribed to in the startup event of the application, so the exact location changes based on the platform you're using to build your application. If it's a WinUI application, you're going to use the **OnLaunched()** event declared in the **App** class, as in the following example:

```
protected override void OnLaunched(Microsoft.UI.Xaml.  
    LaunchActivatedEventArgs args)  
{  
    ToastNotificationManagerCompat.OnActivated  
        += eventArgs =>  
    {  
        //manage the activation  
    };  
    var m_window = new MainWindow();  
    m_window.Activate();  
}
```

If it's a WPF application, instead, you use the **OnStartup()** event exposed by the **App** class, as in the following example:

```
public partial class App : Application  
{  
    protected override void OnStartup(StartupEventArgs  
e)  
    {  
        ToastNotificationManagerCompat.OnActivated  
            += eventArgs =>  
        {  
            //manage the activation  
        };  
    }  
}
```

Regardless of the platform of your choice, whenever the user interacts with a notification your application will be launched and the **OnActivated** event will be triggered. The heart of the activation is the event arguments, which you can use to retrieve the arguments you have set when you have created the notification. The way you retrieve the arguments is the same regardless of whether they are associated with the notification click or to the interaction with specific control, like a button. Let's start with the following notification, which supports both scenarios:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .AddArgument("action", "click")
        .AddArgument("messageId", "1983")
        .AddButton(new ToastButton()
            .SetContent("Mark as read")
            .AddArgument("action", "read")
            .AddArgument("messageId", "1983"))
        .AddButton(new ToastButton()
            .SetContent("Flag as important")
            .AddArgument("action", "flag")
            .AddArgument("messageId", "1983"))
        .Show();
}
```

In this example, we have three types of interaction: through the click on the notification, through the **Mark as read** button, and through the **Flag as important** button. Each of them returns a different action as an argument, in addition to the **messageId** with the identifier of the message. The following code shows how we can handle this notification in the **OnActivated()** event:

```
protected override void OnLaunched(Microsoft.UI.Xaml
    .LaunchActivatedEventArgs args)
{
    ToastNotificationManagerCompat.OnActivated
        += eventArgs =>
    {
        ToastArguments args = ToastArguments.Parse
            (eventArgs.Argument);
        string action = args.Get("action");
        int messageId = args.GetInt("messageId");
        switch (action)
```

```
{
    case "click":
        //display the message
        break;
    case "read":
        //mark the message as read
        break;
    case "flag":
        //mark the message as flagged
        break;
}
};
var m_window = new MainWindow();
m_window.Activate();
}
```

Thanks to the **ToastArguments** class, we can use the **Parse()** method to parse the activation arguments (stored in the **Argument** property) and work with them in an easier way. Specifically, the **ToastArguments** class exposes multiple methods to retrieve the value of an argument already converted into the expected type. You can see two examples in the code:

- The value of the **action** argument is a **string**, so we use the **Get()** method to retrieve it.
- The value of the **messageId** argument is a number, instead, so we use the **GetInt()** method to retrieve it.

There are many other methods available, such as **GetBoolean()**, **GetFloat()**, or **GetEnumerator()**.

Thanks to the notification arguments, we can perform the right task based on the action selected by the argument. For example, if the action is **click**, we could use the **messageId** to retrieve the specific message and display it to the user; or if the action is **read**, we could use the

messageId to find the message in the database and change the field that contains the information if the message has been read.

We have just scratched the surface in this section. Buttons and clicks aren't the only ways we can use to add interactivity. Let's see some more advanced ways.

Supporting user input

Thanks to notifications, we can ask for input from our users without needing to open the application, thanks to the built-in support for text boxes and dropdowns. In the activation arguments, we can retrieve this input, even if we'll need to use some different APIs than the one, we have seen to retrieve the activation arguments.

Let's start with the text box scenario with the following example:

```
private void SendNotification()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .AddInputTextBox("reply", "Type your message",
            "Message")
        .AddButton(new ToastButton()
            .SetContent("Send")
            .AddArgument("action", "send"))
        .SetTextBoxId("reply"))
        .Show();
}
```

To add a text box you can use the **AddInputTextBox()** method. The only required parameter is the identifier of the field (**reply**, in our example), which we'll need during the activation to retrieve the text inserted by the user. The other two are optional: the second is a place-

holder text, which is displayed inside the field until the user types something; the third is a label that is displayed above the field.

When we add user input to a notification, we always need a button to trigger the action, otherwise, the notification will permanently stay on screen, waiting for interaction from the user. For this reason, we also add a button with the **Send** label. We also use the **SetTextBoxId()** method, passing as a parameter the identifier of the field to link the two controls. It's not required, but it's highly recommended: thanks to this setting, the button will be displayed at the right of the **TextBox**; additionally, the button will be disabled until the user starts to type something in the field.

This is the notification generated by the previous code:

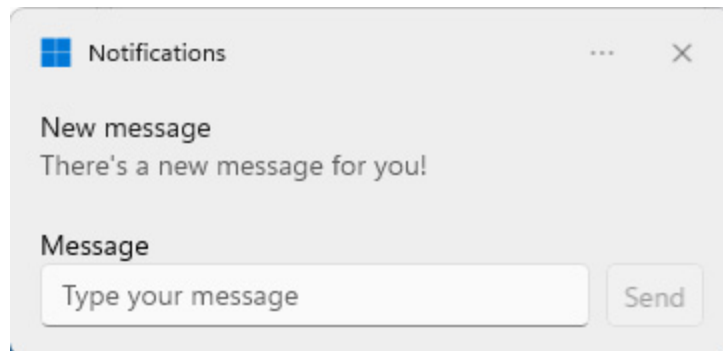


Figure 9.6 – A toast notification with a TextBox as input field

To handle the activation, we still need to use the **OnActivated()** event exposed by the **ToastNotificationManagerCompat** class. However, the user input won't be passed as an argument, but to a special collection called **UserInput**. Let's take a look at the following example:

```
protected override void OnLaunched(Microsoft.UI.Xaml
    .LaunchActivatedEventArgs args)
{
    ToastNotificationManagerCompat.OnActivated
        += eventArgs =>
    {
        ToastArguments args = ToastArguments.Parse
```

```
        (eventArgs.Argument);
        string action = args.Get("action");

        var input = eventArgs.UserInput;
        string message = input["reply"].ToString();
        Console.WriteLine($"Action {action} with
message
        {message}");
    };
    var m_window = new MainWindow();
    m_window.Activate();
}
```

UserInput is a collection of all the inputs provided by the user, categorized by the identifier we have assigned to the input control. In our scenario, we use the keyword **reply** as the identifier, which we have set when we have created the notification. As a value, we'll get in return the message typed by the user.

Another type of user input is the dropdown, which we can add to a notification using the **AddToastInput()** method and the **ToastSelectionBox** class, as in the following example:

```
private void OnSendNotificationWithDropdown ()
{
    new ToastContentBuilder()
        .AddText("New message")
        .AddText("There's a new message for you!")
        .AddToastInput(new ToastSelectionBox("response")
        {
            DefaultSelectionBoxItemId = "ok",
            Items =
            {
                new ToastSelectionBoxItem("ok", "Ok"),
                new ToastSelectionBoxItem("late", "I'll be
late"),
            }
        })
    }
```

```
        new ToastSelectionBoxItem("thanks",  
    "Thanks!")  
    }  
    })  
    .AddButton(new ToastButton()  
        .SetContent("Ok")  
        .AddArgument("action", "send"))  
    .Show();  
}
```

When we create a **ToastSelectionBox** object, we must initialize it with the field identifier, which we'll need later to retrieve the selected value in the activation. Then we can define the available choices through the **Items** collection, which contains multiple **ToastSelectionBoxItem** objects. Each of them is created with a key (which we'll receive in the activation to understand which option was selected) and a text, which will be displayed in the notification. We can also use the **DefaultSelectionBoxItemId** property to specify the identifier of the item that we want to use as default. Also, in this case, we must have at least one button to trigger the interaction.

This is how the notification looks like:

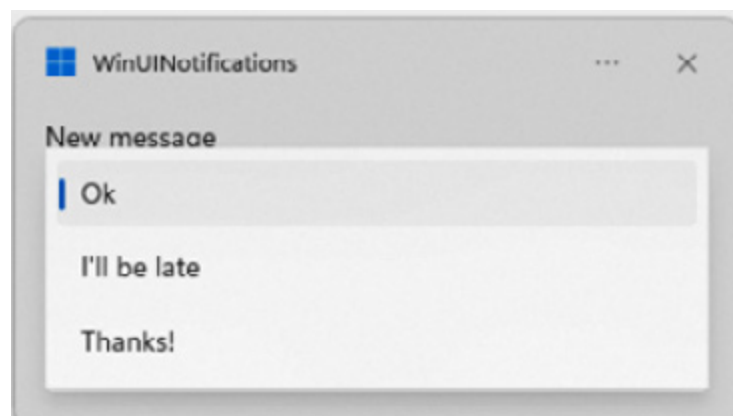


Figure 9.7 – A toast notification with a dropdown as user input

The way we retrieve the selected item in the activation is the same we have seen with the text field: through the **UserInput** collection exposed by the activation arguments.

```
protected override void OnLaunched(Microsoft.UI.Xaml
    .LaunchActivatedEventArgs args)
{
    ToastNotificationManagerCompat.OnActivated
        += eventArgs =>
    {
        ToastArguments args = ToastArguments.Parse
            (eventArgs.Argument);
        string action = args.Get("action");

        var input = eventArgs.UserInput;
        string message = input["response"].ToString();
        switch (message)
        {
            case "ok":
                //send ok message
                break;
            case "late":
                //send late message
                break;
            case "thanks":
                //send thanks message
                break;
        }
    };
    var m_window = new MainWindow();
    m_window.Activate();
}
```

In this case, the key we use to get the value from the **UserInput** collection is the identifier we have used to create the **ToastSelectionBox** object (**response**, in our case). As a value, we'll get the identifier of the option selected by the user, which we can use the way we prefer; in the sample, we'll compose and send a different predefined response, based on the selection of the user.

The last user input we're going to explore is snooze/dismiss, which is a great fit for scenarios where the notification is used as a reminder. Thanks to this option, we can easily enable the user to snooze the notification, so that the reminder will be postponed for later.

To create such a notification, it's enough to set some extra properties:

```
private void OnSendNotificationWithReminder ()
{
    new ToastContentBuilder()
        .SetToastScenario(ToastScenario.Reminder)
        .AddText("It's time!")
        .AddText("This is your reminder")
        .AddButton(new ToastButtonSnooze())
        .AddButton(new ToastButtonDismiss())
        .Show();
}
```

Compared to a traditional notification, we have made the following changes:

- We call the **SetToastScenario()** method, passing as a parameter the **Reminder** value of the **ToastScenario** enumerator. With this setting, the notification will stay on the screen until the user has snoozed or dismissed it. Another possibility is to use the **Alarm** value, which enables alarm scenarios. In this case, the notification will also have a looping sound, which will play until the user acts.
- We add the snooze and dismiss buttons but, instead of using the generic **ToastButton** class, we use the specific **ToastButtonSnooze** and **ToastButtonDismiss** ones. This way, Windows will automatically apply the correct behavior: the snooze button will postpone the reminder, while the dismiss one will make the notification disappear permanently.

In this scenario, you don't need to manually handle the activation, since Windows will take care of everything for you.

With this standard configuration, the snooze will use a default timer set by Windows. However, you can use a **ToastSelectionBox** to give the option to the user to choose after how much time the notification should appear again, as we have done in the following example:

```
private void OnSendNotificationWithReminder ()
{
    new ToastContentBuilder()
        .SetToastScenario(ToastScenario.Reminder)
        .AddText("It's time!")
        .AddText("This is your reminder")
        .AddButton(new ToastButtonSnooze() { SelectionBoxId
=
    "snoozeTime" })
        .AddButton(new ToastButtonDismiss())
        .AddToastInput(new ToastSelectionBox("snoozeTime")
        {
            DefaultSelectionBoxItemId = "5",
            Items =
            {
                new ToastSelectionBoxItem("5", "5
minutes"),
                new ToastSelectionBoxItem("15", "15
minutes"),
                new ToastSelectionBoxItem("30", "30
minutes"),
                new ToastSelectionBoxItem("60", "1 hour"),
            }
        })
        .Show();
}
```

The way you set up the **ToastSelectionBox** object is the same we have previously seen for a regular dropdown. The only difference is that, through the **SelectionBoxId** property, we connect the **ToastButtonSnooze** control with the **ToastSelectionBox**, by passing

the same identifier. This way, Windows will automatically handle the snoozing and the notification will be delayed for the time selected by the user. The time must be passed as the first parameter of each **ToastSelectionBoxItem**, expressed in minutes.

In this section, we have explored all the features that we can include in a notification, from the basic ones (such as text and image) to the most advanced ones (such as buttons and dropdowns). Let's move on now and see another advanced scenario: displaying the status of an operation.

Displaying a progress bar

A common requirement for applications is to notify the user about the progress of an operation. What if you want to keep the user up to date even when the application is minimized? Notifications can help you by integrating a progress bar that the main application can update when needed.

This scenario is supported by the concept of bindable data, which enables us to add one or more placeholders in the notification, which can be replaced at a later stage with actual data.

Let's see first how we can create such a notification:

```
string tag = "report-progress";
new ToastContentBuilder()
    .AddText("Report generation in progress...")
    .AddVisualChild(new AdaptiveProgressBar()
    {
        Title = "Report generation",
        Value = new BindableProgressBarValue
            ("progressValue"),
        ValueStringOverride = new
```

```
        BindableString("progressValueString"),
        Status = "Generating report..."
    })
    .Show(toast =>
    {
        toast.Data = new NotificationData();
        toast.Data.Values["progressValue"] = "0.0";
        toast.Data.Values["progressValueString"] = "0
%";
        toast.Data.SequenceNumber = 1;
        toast.Tag = tag;
    });
```

This time we're using a new method called **AddVisualChild()**, which enables us to add arbitrary elements to the notification. The object we create is called **AdaptiveProgressBar** that, other than simple information such as **Title** and **Status**, exposes two special properties:

- **Value:** This is a number between 0 and 1 that gets translated with the progress bar filling. This property is set with a **BindableProgressBarValue** object identified by the **progressValue** key. Thanks to this object, we'll be able to update the value even after the notification has been generated and displayed.
- **ValueStringOverride:** This is a text that is displayed right below the progress bar. It's typically used to provide a more descriptive status of the operation, such as a percentage or the number of processed items against the total. Since it's a text, we set the property with a **BindableString** object, identified by the **progressValueString** key. Also, in this case, we'll be able to update the text after the notification has been generated.

When we call the **Show()** method, we used the same approach we have learned before to further customize the toast notification. One of the fields should be familiar: it's the **Tag** property, which we'll need later to identify the notification that we must update.

The other field is new, and it's called **Data**. Its type is **NotificationData** and we can use it to replace the placeholders that we have included in the notification's template with actual data. The placeholders are stored inside the **Values** collection, and they are identified by the keys we have set in the template. When the notification is generated, it's likely that the operation we want to track with a progress bar hasn't started yet, so we set both values to **0**. Then, we also set the **SequenceNumber** property to **1**, to notify Windows that this is the first update.

If we execute this code, the notification will be displayed and it will stay visible, waiting for further updates. Let's see now how we can track progress by analyzing the following example:

```
for (uint cont = 0; cont <= 10; cont++)
{
    await Task.Delay(1000);
    var data = new NotificationData
    {
        SequenceNumber = cont
    };
    IFormatProvider provider =
        CultureInfo.CreateSpecificCulture("en-US");
    double progressValue = (double)cont / 10;
    string progressValueConverted = $"{(progressValue *
        100).ToString()} %";
    data.Values["progressValue"] =
        progressValue.ToString(provider);
    data.Values["progressValueString"] =
        progressValueConverted;
    ToastNotificationManagerCompat.CreateToastNotifier(
    )
        .Update(data, tag);
}
```

We are simulating a long-running operation using a **for** statement, which will increment the counter from 1 to 10, with a 1-second delay at each cycle. At every execution, we create a new **NotificationData** object, and we do the same thing we did when we have created the notification: we update the **SequenceNumber** property and the two placeholders, **progressValue**, and **progressValueString**, through the **Values** collection. The difference is that this time, we're incrementing the progress bar, using the counter as a value. The only special scenario to handle is that the **BindableProgressBarValue** object requires a value from 0 (the progress bar is empty) to 1 (the progress bar is full), as such we must convert the counter into a percentage.

Once we have populated the **NotificationData** object, we can update the existing notification through the **Update()** method exposed by the **ToastNotifier** object created with the **ToastNotificationManagerCompat** class. Other than the data, we must also pass the tag of the notification, so that Windows can identify which is the notification to update. Refer to the following screenshot:

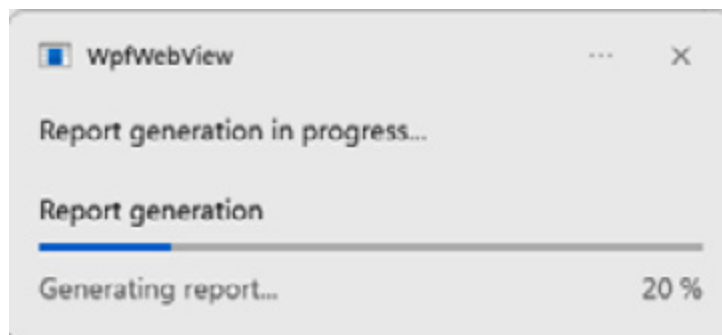


Figure 9.8 – A notification with a progress bar

The previous code is a simplification to help you learn the basic concepts of this feature. In a real application, the progress would be calculated out of a real operation, like downloading a file from the internet, loading data into a database, and so on.

With progress bars, we have explored all the opportunities that toast notifications give you to enhance the experience of your users. Now

it's time to see another type of notifications supported by Windows: badge notifications.

Displaying a badge in the taskbar

If your application is packaged with MSIX, you can support it with another type of notification, called **badge notifications**. These notifications are helpful when the user decides to pin your application on the Windows taskbar: the badge, in fact, is displayed in an overlay of your application's icon. Badge notifications support two kinds of content: a number (for example, you want to quickly display the number of unread messages) or a symbol (for example, you want to notify users that the application needs their attention).

To use these type of notifications, other than the **CommunityToolkit.WinUI.Notifications** namespace, you must also add the **Windows.UI.Notifications** class.

Let's start with an example of creating badge notifications with a number:

```
private void OnSendBadgeNumberNotification ()
{
    BadgeNumericContent numeric = new
        BadgeNumericContent(15);
    var xml = numeric.GetXml();
    BadgeNotification notification = new
        BadgeNotification(xml);
    var badgeManager = BadgeUpdateManager.CreateBadge
        UpdaterForApplication();
    badgeManager.Update(notification);
}
```

Also, in this case, the Windows Community Toolkit offers a class that we can use to create badge numeric notifications without having to

manually write the XML. We just need to create a new **BadgeNumericContent** object passing, as a parameter, the number we want to display. However, we don't have an equivalent of the **ToastContentBuilder** class, so to display the notification we must use the standard UWP classes **BadgeNotification** and **BadgeUpdateManager**, which can manipulate only the XML content.

To handle this scenario, the toolkit provides a helper method called **GetXml()** that converts the C# object into XML. Once we have it, we can wrap it inside a **BadgeNotification** object and pass it as a parameter to the **Update()** method exposed by the **BadgeUpdateManager** class.

Badge notifications with a symbol (called **glyph**) work in the same way, except that we use the **BadgeGlyphContent** class instead of the **BadgeNumericContent** one. In this case, when we initialize the object, we must pass one of the values of the **BadgeGlyphValue** enumerator, which represents different types of symbols. The following example shows an alert badge on top of the application's icon:

```
private void OnSendBadgeGlyphNotification ()
{
    BadgeGlyphContent glyph = new
        BadgeGlyphContent(BadgeGlyphValue.Alert);
    var xml = glyph.GetXml();
    BadgeNotification notification = new
        BadgeNotification(xml);
    var badgeManager = BadgeUpdateManager.CreateBadge
        UpdaterForApplication();
    badgeManager.Update(notification);
}
```

The following screenshot shows the two types of badge notifications in action:

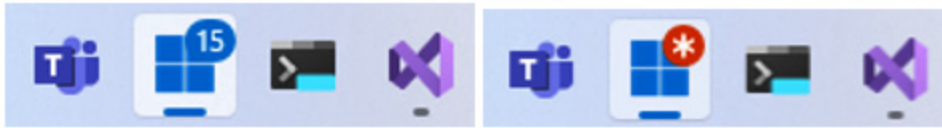


Figure 9.9 – A numeric badge notification (on the left) and a glyph badge notification (on the right)

With this we covered one set of notifications, now let's move to the final section of this chapter which will be quite brief: as mentioned at the beginning of the chapter, we're going to talk about push notifications, but only as an overview.

Implementing push notifications

All the scenarios we have seen so far are based on local notifications, which are generated directly by your application when a specific event happens. However, there are scenarios where the event isn't generated locally, but remotely. A new message, a new mail, a new goal scored by your favorite football team are just a few examples. Sure, your application could continuously send messages to your backend to check if there are updates, but it wouldn't be very efficient. This approach would have a huge impact on the battery and the resources of your computer.

Therefore, the industry has introduced the concept of push notifications. Instead of the device continuously checking with the backend if there are updates, it's the backend itself that sends a message to the device whenever there's something new. This scenario is made possible thanks to the introduction of a cloud service, which is provided by the owner of the platform and acts as a middleman between the client application and the backend and enables the following flow:

1. The user registers their application against the cloud service, and they receive back a client ID and a client secret that they can use to perform authentication.

2. The first time the application starts, it contacts the cloud service to register the device and it receives back a channel, represented by an HTTP URL.
3. The application sends the HTTP URL to the backend so that it can store it in a database or another similar solution. The channel is stored with some additional information that the backend can use to identify the user. We don't want to send a notification about a message to the wrong user, or an update about the football match to people who aren't interested in that particular team.
4. Whenever there's a new notification to send, the backend authenticates against the cloud service and, if it's successful, it receives an authentication token.
5. The backend generates a payload with the content of the notification and sends it to the HTTP URL of the device, together with the authentication token.
6. If the token is valid, the cloud service processes the notification, and it dispatches it to the device which corresponds to the HTTP URL.
7. The device receives the message, and it uses the payload to generate a notification.

This architecture is adopted by all the players on the market who build mobile and desktop ecosystems: from Microsoft to Google to Apple. However, each platform has its own specific implementation of the following three key areas:

- The authentication system against the cloud service
- The payload's format (some services use XML, some other JSON)
- The type of notifications that are supported by the operating system

The Microsoft implementation of this cloud service is called **Windows Notification Service (WNS)**, which supports three types of notifications:

- Toast notifications
- Badge notifications
- Raw notifications

Technically, WNS supports a fourth type, which is **tile** notifications, but since they aren't supported anymore by Windows 11, we aren't considering them in this section.

The first two types don't require any special configuration from the application's side, other than eventually subscribing to the **OnActivated** event exposed by the **ToastNotificationManagerCompat** class to handle the activation. Your backend will send an XML payload, which describes the content of the notification, and Windows will handle it for you, by displaying a toast notification or by adding a badge to your application's icon.

Raw notifications, instead, give you the maximum flexibility since they can contain any arbitrary payload. However, Windows does not natively manage them, but it's up to your application to handle them most appropriately through the activation events provided by the push notification APIs. Raw notifications are used when the message doesn't necessarily need to be translated with visual content. For example, a raw notification can trigger a background sync, or a long-processing task performed by the client application.

But how can we send a notification from our backend?

Implementing the backend

Push notifications are based on a standard technology: it's an XML payload sent to an HTTP URI using a **POST** command. This means that it doesn't matter if your backend is implemented in .NET, Java, or Node.js, you will be able to easily integrate the sending of push notifications.

However, before sending a notification, you must authenticate against the WNS. Without this requirement, any malicious developer would be able to send notifications on your behalf just by spoofing the URI associated with a device. The authentication part is the most relevant new feature introduced by the Windows App SDK. Push notifications, in fact, are already supported by UWP applications and by MSIX packaged Win32 apps. However, there's a strict requirement to satisfy: the application must be published on the Microsoft Store. By registering your application with the Microsoft Store, you are granted the client identifier and the client secret that are needed by your backend to authenticate against the WNS. These identifiers and secrets, however, are tight to the identity assigned to your application from the Store, which means that the only way for your application to receive notifications is to publish it on the Store. Any other distribution technique would lead to the creation of a different identity, preventing the application to receive notifications.

This approach is a blocker for many scenarios (such as enterprise applications) where the application can't be submitted on the Microsoft Store. As such, the Windows App SDK is introducing (still experimental) an updated version of the WNS which, instead of being tied to the Microsoft Store, is connected to **Azure Active Directory (AAD)**. The Azure portal provides a way to register an application that needs to interact with AAD, which is already used today in many scenarios: applications that need to authenticate the user using a Microsoft or Microsoft 365 account; the usage of the Microsoft Graph APIs to integrate your application with the Microsoft 365 ecosystem; and many more. When you register a new application on Azure, you will get the client ID and the client secret that your backend needs to authenticate against WNS, so that it can start to send push notifications. Since AAD isn't tight to the Microsoft Store anymore, this scenario enables the usage of push notifications in any kind of Win32 application, regardless of the way it's distributed.

At the time of writing, however, you must be aware of its limitations.

Limitations of the current implementation

As mentioned at the beginning of the chapter, we aren't going through the topic of push notifications in a detailed way because it's still in pre-view and, as such, subject to multiple changes.

If you want to test this feature, you can follow the guidance available at <https://docs.microsoft.com/en-us/windows/apps/windows-app-sdk/notifications/push/>, but keep in mind the following limitations:

- Push notifications are supported only by MSIX packaged apps.
- The minimum requirement is Windows 10 version 2004.
- There is no guarantee of latency or reliability.
- There's a limit of 1 million notifications per month.
- You can register an application on Azure, but you can't link it to the **Package Family Name (PFN)** (which defines the identity) of your application directly on the portal. You must provide it through a form (the link for which is available in the documentation) and wait for the team to manually associate the PFN to your application ID in the WNS database.

Push notifications are planned to be released with the Windows App SDK 1.1 (scheduled for Q2 2022), which will remove the current limitations and will streamline the authentication process.

Push notifications can help to level up your application, by adding even more engagement to your applications. In this section, we have learned just the basic concepts, which can be useful to understand the architecture to implement when the feature will be moved out of preview.

Summary

Notifications is one of the areas where a native application has a significant advantage compared to a web application, especially on Windows. Web applications and browsers provide some limited support for notifications, but it's only thanks to the rich Windows ecosystem that you can enable all the features we have seen in this chapter—images, interactions, progress bars, reminders. Thanks to these features, we can build more engaging applications, which can interact with the user even when they aren't actively using them.

However, be careful when you decide to implement them.

Notifications can quickly turn from the most useful feature provided by an application to the most annoying one if they are abused. Don't use notifications for advertising or for delivering information that isn't critical or time sensitive.

In the next chapter, we're going to learn another exciting opportunity that the Windows ecosystem provides for our application: the ability to use **Artificial Intelligence (AI)** and **Machine Learning (ML)** through the computational power of our devices.

Questions

1. Toast notifications are supported only by WinUI applications. True or false?
2. Adding a tag to a toast notification is an essential requirement for Windows to display it. True or false?
3. There are no differences between implementing toast notifications in packaged and unpackaged applications. True or false?